OOP · DETAILED INTERVIEW NOTES

Object-Oriented Programming *Notes*

Every concept defined precisely and explained in depth — the four pillars, inheritance, polymorphism, abstract classes vs interfaces, composition, memory, SOLID and more. With code snippets, tables and rapid-fire Q&A.

18	60+	11	5
SECTIONS	DEFINITIONS	Q&A COVERED	SOLID RULES

CONTENTS

01 Introduction to OOP
02 Class & Object
03 Encapsulation
04 Abstraction
05 Inheritance
06 Polymorphism
07 Overloading vs Overriding
08 Abstract Class vs Interface
09 Composition vs Inheritance
10 Constructors & Destructors
11 Static, Final, This, Super
12 Memory Management
13 Object Class Methods (Java)
14 SOLID Principles
15 Exception Handling
16 OOP in Java vs C++ vs Python
17 Common OOP Interview Questions
18 Quick Revision Cheat-Sheet
19 Follow-up Drills

01 Introduction to OOP

DEFINITION

Object-Oriented Programming (OOP) is a programming paradigm that organises software around **objects** — self-contained entities that bundle **data (attributes)** and the **operations (methods)** that act on that data — rather than around functions and logic.

A **paradigm** is simply a style or way of thinking about how to structure a program. Procedural programming organises code as a sequence of steps (functions); OOP organises it as a collection of interacting objects that model real-world entities.

	PROCEDURAL	OBJECT-ORIENTED
Basic unit	Function	Object (class instance)
Data	Global / loosely attached to functions	Encapsulated inside objects
Approach	Top-down (break the problem into steps)	Bottom-up (build from objects)
Reuse	Limited	Inheritance & composition
Examples	C, Pascal, Fortran	Java, C++, Python, C#

Why OOP is used

- **Modularity** — the code is broken into independent objects that can be developed and tested separately.
- **Reusability** — inheritance and composition let you reuse existing code.
- **Maintainability** — changes are localised to the relevant class.
- **Scalability** — large systems are easier to grow.
- **Security** — encapsulation hides internal data.
- **Real-world modelling** — objects map naturally to entities like `User`, `Order`, `Car`.

> THE FOUR PILLARS
>
> Everything in OOP rests on four ideas: **Encapsulation**, **Abstraction**, **Inheritance** and **Polymorphism**.

02 Class & Object

> DEFINITION – CLASS
>
> A **class** is a user-defined **blueprint** or template that defines the attributes (data) and methods (behaviour) common to all objects of a certain kind. It is a logical entity and occupies no memory by itself.

DEFINITION — OBJECT

An **object** is a runtime **instance** of a class. It has a **state** (the current values of its attributes), a **behaviour** (its methods), and an **identity** (a unique reference that distinguishes it from other objects).

ANALOGY

A class is like the architectural *plan* of a house; objects are the actual houses built from that plan. Many houses (objects) can be built from one plan (class), each with its own colour (state).

Anatomy of a class

```
class Car {
  String brand;            // attribute (instance variable)
  int speed;
  static int count;        // class variable (shared)

  Car(String b) {          // constructor
    brand = b; count++;
  }
  void accelerate() {      // method (behaviour)
    speed += 10;
  }
}

Car c1 = new Car("Honda");   // object 1
Car c2 = new Car("Tesla");   // object 2
c1.accelerate();
```

MEMBER	MEANING
Instance variable	Belongs to each object separately (each car has its own `speed`).
Static variable	Belongs to the class, shared by all objects (`count`).
Instance method	Operates on a specific object's state.
Static method	Belongs to the class; cannot access instance variables directly.

03 Encapsulation

DEFINITION

Encapsulation is the bundling of data (attributes) and the methods that operate on that data into a single unit (a class), while **restricting direct access** to the internal state. This restricted access is called **data hiding**.

Access modifiers

MODIFIER	ACCESSIBLE FROM
private	Only within the same class.
protected	Same class and subclasses (and same package in Java).
public	Anywhere in the program.
default (Java)	Anywhere within the same package.

Getters & setters

Fields are kept `private`; controlled access is provided through **getters** (read) and **setters** (write), which can validate the input.

```
private double balance;
public double getBalance() { return balance; }
public void deposit(double amt) {
  if (amt > 0) balance += amt;   // validation
}
```

BENEFITS

Prevents invalid states, improves security, hides implementation so internals can change freely, and makes the class easier to maintain.

04 Abstraction

DEFINITION

Abstraction is the process of **hiding the implementation details** and showing only the essential features of an object to the user. You focus on *what* an object does, not *how* it does it.

It is achieved using **abstract classes** and **interfaces** — you program against a contract rather than a concrete implementation.

REAL-WORLD EXAMPLE

When you drive a car, you use the steering wheel, accelerator and brakes — the essential interface. You don't need to know how the engine, fuel injection or transmission work internally. That is abstraction.

```
interface Payment { void pay(double amt); }   // contract only
class UPI implements Payment {
  public void pay(double amt) { /* UPI internals hidden */ }
}
```

Abstraction vs Encapsulation

	ENCAPSULATION	ABSTRACTION
Hides	Data (state)	Implementation complexity
Purpose	Protect the object's state	Simplify what the user sees
Achieved by	Access modifiers	Abstract classes & interfaces
Answers	"How is data protected?"	"What is exposed?"

TOGETHER

Encapsulation hides the *data*; abstraction hides the *complexity*. They are complementary, not alternatives.

05 Inheritance

DEFINITION

Inheritance is a mechanism by which one class (the **subclass** or child) acquires the attributes and methods of another class (the **superclass** or parent), establishing an **IS-A** relationship. It enables code reuse and polymorphism.

TYPE	MEANING	EXAMPLE
Single	One parent, one child	`Dog extends Animal`
Multilevel	Grandparent → parent → child	`Puppy extends Dog extends Animal`
Hierarchical	One parent, many children	`Car`, `Bike` extend `Vehicle`
Multiple	One child, many parents	C++ `class C : A, B` (Java via interfaces)
Hybrid	A combination of the above	Multilevel + hierarchical

```
class Animal {
  void eat() { System.out.println("eating"); }
}
class Dog extends Animal {     // IS-A Animal
  void bark() { System.out.println("woof"); }
}
Dog d = new Dog();
d.eat();   // inherited from Animal
d.bark();
```

The diamond problem

With multiple inheritance, if two parent classes share a common base, the child inherits **two copies** of that base — creating ambiguity about which one to use.

- **C++** solves it with **virtual inheritance** (only one shared copy of the base).
- **Java** avoids it by allowing only *single* class inheritance — multiple behaviour comes from **interfaces**.

SUPER KEYWORD

`super` refers to the parent class — used to call the parent's constructor or an overridden method.

06 Polymorphism

DEFINITION

Polymorphism (Greek: "many forms") is the ability of a single interface to represent different underlying implementations — the same method call behaves differently depending on the actual object it is invoked on.

	COMPILE-TIME (STATIC)	RUN-TIME (DYNAMIC)
Achieved by	Method **overloading**	Method **overriding** + virtual functions
Binding	Early (at compile time)	Late (at run time)
Decided by	The reference type / arguments	The actual object type
Example	`add(int,int)` vs `add(double,double)`	`Shape s = new Circle(); s.area();`

```
class Shape { double area() { return 0; } }
class Circle extends Shape {
  double area() { return 3.14 * r * r; }   // override
}
Shape s = new Circle();   // parent ref, child object
s.area();                    // Circle's version runs (runtime)
```

UNDER THE HOOD

Runtime polymorphism uses a **vtable** (virtual method table): each object points to a table of function pointers, and the correct method is resolved when the call happens.

07 Overloading vs Overriding

DEFINITION — OVERLOADING

Method overloading means defining multiple methods with the **same name** but **different parameter lists** (different number, type or order of parameters) within the same class. It is resolved at **compile time**.

DEFINITION — OVERRIDING

Method overriding means redefining a parent class method in a subclass with the **same name, parameters and return type**, replacing its behaviour. It is resolved at **run time**.

	OVERLOADING	OVERRIDING
Where	Same class	Parent & child class
Parameters	Must differ	Must be the same
Return type	Can differ	Same (or a covariant subtype)
Binding	Compile-time (static)	Run-time (dynamic)
Purpose	Convenience / flexibility	Specialise behaviour

```
// Overloading
int add(int a, int b) { ... }
double add(double a, double b) { ... }

// Overriding
class Animal { void sound() { print("..."); } }
class Dog extends Animal { void sound() { print("woof"); } }
```

OVERRIDING RULES

You cannot reduce visibility (public → private), cannot override `final` / `static` /private methods, and should annotate with `@Override` in Java to catch mistakes.

08 Abstract Class vs Interface

DEFINITION — ABSTRACT CLASS

An **abstract class** is a class declared with the `abstract` keyword that **cannot be instantiated**. It may contain both abstract methods (declared, no body) and concrete methods (with a body), plus fields/state.

DEFINITION — INTERFACE

An **interface** is a reference type that defines a **contract** — a set of method signatures that any implementing class must provide. It specifies *what* to do, never *how*.

	ABSTRACT CLASS	INTERFACE
Methods	Abstract + concrete	Abstract (Java 8+ allows default/static)
State	Can have instance fields	Only constants (traditionally)
Inheritance	Single (`extends` one)	Multiple (`implements` many)
Constructor	Yes	No
When to use	Related classes sharing code (IS-A)	Unrelated classes sharing a capability (CAN-DO)

RULE OF THUMB

Use an **abstract class** when subclasses are closely related and share code; use an **interface** for a capability that unrelated classes can implement (`Comparable` , `Serializable` , `Runnable`).

09 Composition vs Inheritance

DEFINITION — COMPOSITION

Composition is a design technique where a class contains an instance of another class as a member, establishing a **HAS-A** relationship ("a Car *has an* Engine").

	INHERITANCE (IS-A)	COMPOSITION (HAS-A)
Relationship	"A Car **is a** Vehicle"	"A Car **has an** Engine"
Coupling	Tight — child depends on parent internals	Loose — parts are independent
Flexibility	Fixed at compile time	Can swap parts at run time

DESIGN PRINCIPLE

Favour composition over inheritance. Deep inheritance hierarchies become fragile (the "fragile base class" problem); composition keeps classes small, testable and swappable.

The three object relationships

RELATIONSHIP	MEANING	EXAMPLE
Association	Objects know / use each other	Teacher – Student
Aggregation	Weak HAS-A; the part can outlive the whole	Department – Professor
Composition	Strong HAS-A; the part dies with the whole	House – Room

10 Constructors & Destructors

DEFINITION — CONSTRUCTOR

A **constructor** is a special method that is called **automatically when an object is created**, used to initialise the object's state. It has the **same name as the class** and **no return type**.

DEFINITION — DESTRUCTOR

A **destructor** is a special method called automatically when an object is destroyed, used to release resources (memory, file handles). In C++ it is `~ClassName()` ; Java and Python rely on **garbage collection** instead.

TYPE	MEANING
Default	No arguments; auto-created if you define none.
Parameterised	Takes arguments to set initial values.
Copy	Creates a new object from an existing one.

```
class Point {
  int x, y;
  Point() { x = 0; y = 0; }              // default
  Point(int a, int b) { x = a; y = b; } // parameterised
  Point(Point p) { x = p.x; y = p.y; }   // copy
}
```

Deep vs Shallow copy

	SHALLOW COPY	DEEP COPY
Copies	References (both share the same inner objects)	All nested objects (fully independent)
Risk	Changing one affects the other	Safer, but more expensive

> **ORDER & CHAINING**
>
> Construction runs **parent → child**; destruction runs **child → parent**. **Constructor chaining** (`this()` / `super()`) calls another constructor of the same or parent class.

11 Static, Final, This, Super

static

Belongs to the **class** rather than any instance — shared by all objects. Used for counters, constants and utility methods. A static method cannot use `this` or instance variables.

final (Java) / const (C++)

Means "cannot be changed". A `final` *variable* is a constant, a `final` *method* cannot be overridden, and a `final` *class* cannot be extended.

this

A reference to the **current object**. Used to resolve name clashes between a parameter and a field, and to call another constructor (`this()`).

super

A reference to the **parent class**. Used to call the parent's constructor (`super()`) or an overridden method (`super.method()`).

EXAMPLE

A `static int count` in a class counts how many objects have been created — because it is shared across all instances, not duplicated per object.

12 Memory Management

	STACK	HEAP
Stores	Local variables & method call frames	Objects
Speed	Very fast	Slower
Lifetime	Until the method returns	Until garbage collected / freed
Managed by	Automatically	GC (Java/Python) or `new`/`delete` (C++)

DEFINITION — GARBAGE COLLECTION

Garbage collection is the automatic reclamation of heap memory that is no longer referenced by any live object. Java, Python and C# do this automatically; C++ does not.

Value types (int, char, bool) hold the data directly; **reference types** (objects) hold a reference to data on the heap.

PITFALLS

Memory leak — memory allocated but never freed (common in C++). **Dangling pointer** — a pointer to already-freed memory. Garbage collection removes both risks for managed languages.

13 Object Class Methods (Java)

Every class in Java implicitly extends `Object`, which provides methods you are expected to know and often override.

METHOD	PURPOSE
`toString()`	Textual representation of the object.
`equals(Object)`	Logical equality (compare contents).
`hashCode()`	Integer hash for hash-based collections.
`getClass()`	Runtime class of the object.
`clone()`	Creates a copy of the object.

THE EQUALS/HASHCODE CONTRACT

If two objects are `equals()`, they **must** have the same `hashCode()`. Always override both together, or hash-based collections (`HashMap`, `HashSet`) misbehave.

== VS EQUALS()

`==` compares **references** (same object?). `equals()` compares **values** (same content?) — when properly overridden.

14 SOLID Principles

Five principles for clean, extensible object-oriented design.

S — Single Responsibility	A class should have **one reason to change** — one job.
O — Open/Closed	Open for extension, closed for modification — add behaviour without editing existing code.
L — Liskov Substitution	A subclass must be usable anywhere its parent is, without breaking correctness.
I — Interface Segregation	Many small, focused interfaces beat one fat interface; don't force clients to depend on methods they don't use.
D — Dependency Inversion	Depend on abstractions (interfaces), not concrete classes.

EXAMPLES

- **SRP:** an `OrderService` that computes price *and* emails customers → split into `PricingService` and `EmailService`.
- **DIP:** inject a `PaymentGateway` interface instead of a concrete `StripeGateway`.
- **OCP:** add a new payment method as a new class implementing the interface — no edits to existing code.

15 Exception Handling

DEFINITION

An **exception** is an event that disrupts the normal flow of a program. **Exception handling** is the mechanism to detect, signal and recover from such errors gracefully instead of crashing.

```
try {
  int x = 10 / 0;              // may throw
} catch (ArithmeticException e) {
  System.out.println("Divide by zero!");
} finally {
  // always runs — cleanup
}
```

TERM	MEANING
Checked	Must be handled or declared (e.g. `IOException`).
Unchecked	Runtime errors, programmer's fault (e.g. `NullPointerException`).
throw	Raises an exception explicitly.
throws	Declares that a method may throw an exception.
Custom	Extend `Exception` to define your own error type.

finally always executes — used to release resources (closing files/connections).

16 OOP in Java vs C++ vs Python

FEATURE	JAVA	C++	PYTHON
Multiple inheritance	Interfaces only	Yes (virtual base)	Yes (MRO)
Memory	Garbage collected	Manual (new/delete)	Garbage collected
Pointers	No	Yes	No (references)
Virtual methods	Default (override)	Opt-in (`virtual`)	All methods
Default access	package-private	private (class)	public
Interfaces	Yes (`interface`)	Via pure virtual classes	ABCs / duck typing

17 Common OOP Interview Questions

Q. What is OOP, and what are its four pillars?

A paradigm that organises code around objects bundling data and behaviour. Pillars: Encapsulation, Abstraction, Inheritance, Polymorphism.

Q. Class vs Object?

A class is a blueprint (no memory); an object is a runtime instance of it with its own state and identity.

Q. Encapsulation vs Abstraction?

Encapsulation hides *data* (protects state via access modifiers); abstraction hides *complexity* (exposes only essentials via abstract classes/interfaces).

Q. Overloading vs Overriding?

Overloading = same name, different parameters, same class, compile-time. Overriding = same signature in a subclass, run-time.

Q. Abstract class vs Interface?

Abstract class can hold state and concrete methods, single inheritance; an interface is a pure contract, multiple implementation.

Q. Why favour composition over inheritance?

Looser coupling and runtime flexibility; avoids fragile, deep hierarchies.

Q. What is the diamond problem, and how is it solved?

Ambiguity from multiple inheritance when two parents share a base. C++ uses virtual inheritance; Java uses interfaces.

Q. Can a constructor be private?

Yes — used in the Singleton pattern to prevent direct instantiation.

Q. What is the equals()/hashCode() contract?

Objects that are equal must return the same hash code — override both together.

Q. == vs equals()?

`==` compares references; `equals()` compares content (when overridden).

Q. Difference between shallow and deep copy?

Shallow copy shares nested references; deep copy duplicates them, giving independent objects.

18 Quick Revision Cheat-Sheet

- **4 pillars:** Encapsulation, Abstraction, Inheritance, Polymorphism.
- Class = blueprint (no memory); Object = instance (state + behaviour + identity).
- Access: private < protected < public.
- Overloading = compile-time; Overriding = runtime (vtable).
- Abstract class = state + code, single inheritance; Interface = contract, multiple.
- Inheritance = IS-A; Composition = HAS-A; **favour composition**.
- Relationships: association, aggregation (weak), composition (strong).
- Deep copy = independent objects; shallow copy = shared references.
- Stack = locals (fast); Heap = objects (GC).
- SOLID: SRP, OCP, LSP, ISP, DIP.
- equals() and hashCode() must be overridden together.
- Diamond problem → virtual inheritance (C++) / interfaces (Java).

> MOST-ASKED TRIO
>
> **Overloading vs Overriding**, **Abstract class vs Interface**, **Composition vs Inheritance** — master these and you're through most OOP rounds.

OOP · Detailed Interview Notes 18 sections · interview-ready

19 Follow-up Drills

Interviewers rarely stop at the first answer. Each drill below is the main question, then the follow-ups they actually push on — practise the whole chain, not just the first line.

Q Inheritance vs composition — which do you prefer?

↳ **Why is composition usually safer?**

It avoids the fragile base class problem — a change in a parent class can silently break subclasses. Composition depends on a published interface, so changes stay contained.

↳ **When is inheritance genuinely the right choice?**

When there is a true is-a relationship with substitutability: the subtype must be usable anywhere the base type is expected, without surprising behaviour.

Q Abstract class vs interface — how do you choose?

↳ **Which one gives you multiple inheritance of type?**

Interfaces — a class can implement many, which is how Java avoids the diamond problem for state while still allowing multiple contracts.

↳ **Why did Java 8 add default methods to interfaces?**

To add methods to existing interfaces without breaking every implementer — essential for evolving the collections API while keeping binary compatibility.

Q Compile-time vs runtime polymorphism?

↳ **How does runtime dispatch actually work?**

Through a vtable (or equivalent): the object holds a pointer to its class's method table, and the call resolves through that pointer at runtime rather than being fixed at compile time.

↳ **Can you overload by return type alone?**

No — the compiler cannot choose between two methods that differ only in return type, because the call site does not always specify the expected type.

Q Pick a SOLID principle and give a real violation.

↳ **How does the Dependency Inversion Principle help testing?**

By depending on an interface rather than a concrete class, you can inject a fake in tests instead of a database or network client — the code becomes testable without infrastructure.

↳ **What does the Open/Closed Principle mean in practice?**

You add behaviour by adding a new class (a new strategy, a new handler) rather than editing a growing if/else chain — extension without modification.

Q Why is encapsulation more than getters and setters?

↳ How do you expose data read-only?

Return an immutable view or a copy, or expose a method that answers the question rather than the raw field — otherwise callers can mutate internal state.

↳ Why not just make fields public?

Because then every field is part of your public contract; you can never change the representation or enforce an invariant without breaking callers.

Q What causes memory leaks in a garbage-collected language?

↳ Stack vs heap — what lives where?

Stack holds frames and local primitives with automatic lifetime; the heap holds objects whose lifetime the GC manages. Leaks happen on the heap when references are retained unintentionally.

↳ Give a common leak in Java or Python.

Static or long-lived collections that keep growing, listeners never unregistered, or caches without eviction — the object is reachable, so the GC cannot reclaim it.

Q Singleton vs a class with static methods?

↳ Why is Singleton criticised?

It is global mutable state, which makes testing and concurrency hard, and it hides dependencies. Dependency injection is usually the better answer.

↳ How do you make a Singleton thread-safe?

Use an eager-initialised static instance or a holder class (initialisation-on-demand), rather than lazy double-checked locking done incorrectly.

Q How does exception handling affect design?

↳ Checked vs unchecked exceptions?

Checked exceptions force the caller to handle a recoverable condition; unchecked signal programming errors. Overusing checked exceptions pushes boilerplate up the call stack.

↳ Why should you not use exceptions for control flow?

They are expensive (stack capture) and hide the normal path, making code harder to reason about — use return values for expected outcomes.